esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2025-2026 **Applications Web Distribuées**

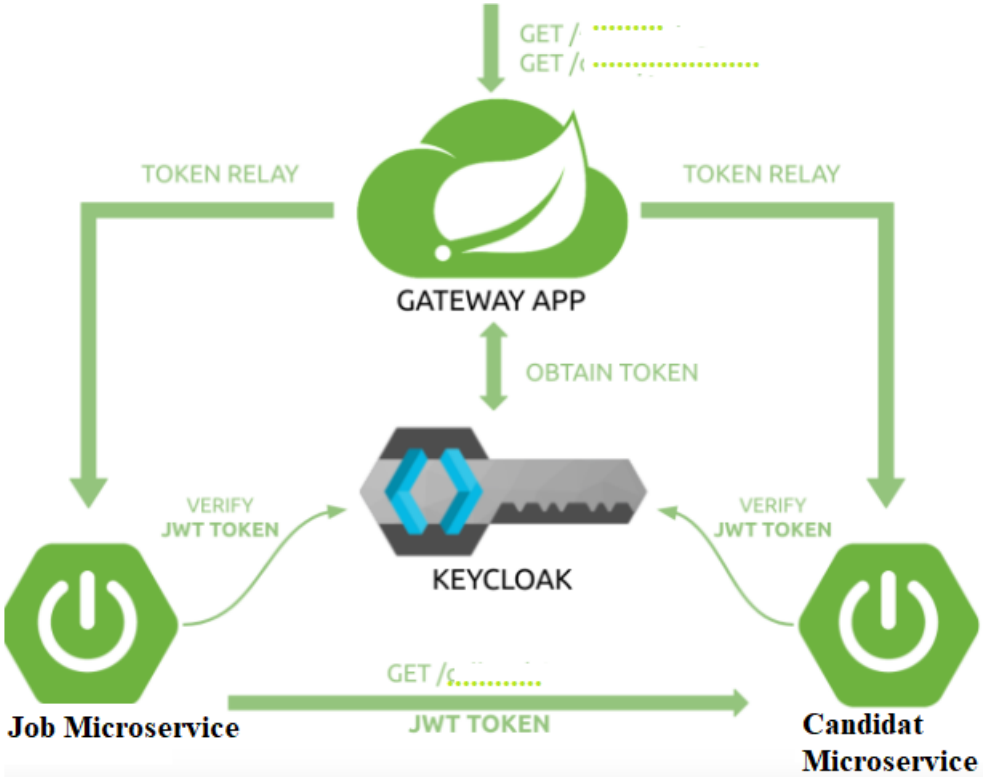
Sécurisation d'une API Gateway avec Keycloak

Objectif

- Assurer la sécurité d'une API Gateway en utilisant Auth2.0.
- Assurer la sécurité de l'accès vers l'ensemble des microservices.

Principe de fonctionnement

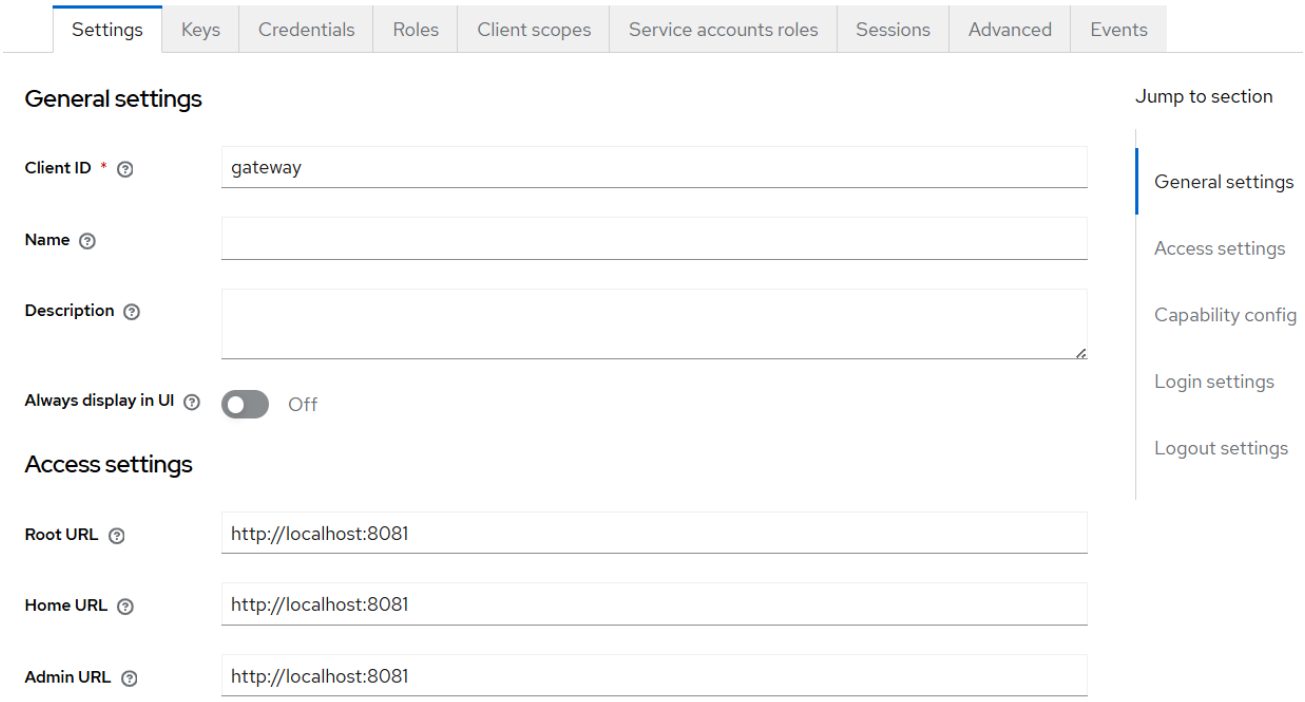
Lors de l'accès aux Microservices à travers le Gateway, celle-ci va renvoyer automatiquement l'utilisateur vers Keycloak pour récupérer un Token. Keycloak authentifiera l'utilisateur si besoin, puis renverra des informations sur l'utilisateur et le fameux Token à notre Viewer. Ce Token sera ensuite utilisé pour l'accès vers le microservice en question.

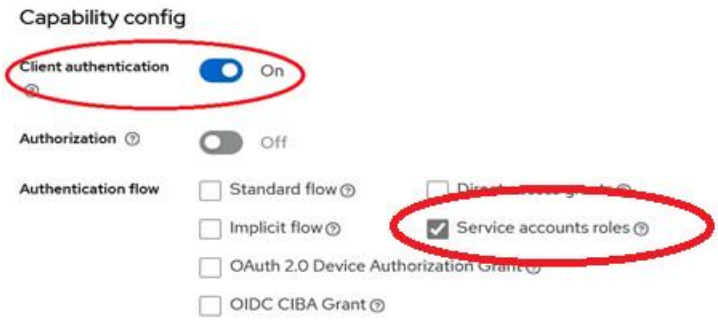


Les étapes à suivre

1. Créer un realm « **JobBoardKeycloack** »
2. Créer un client **gateway**
   - clientID : gateway (nom de l'application Microservice Gateway)
   - RootURL, HomeURL et AdminURL : http://localhost:8081 (Port de MS Gateway)

- Vérifier la configuration ajoutée selon les figures ci-dessous :





3. Dans le fichier pom.xml de votre projet Gateway, ajouter le code suivant :

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>
```

```
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>


    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-oauth2-client</artifactId>
    </dependency>


    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
    </dependency>


  </dependencies>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
```

```
                <configuration>
                  <excludes>
                    <exclude>
                      <groupId>org.projectlombok</groupId>
                      <artifactId>lombok</artifactId>
                    </exclude>
                  </excludes>
                </configuration>
              </plugin>
            </plugins>
          </build>
          <repositories>
            <repository>
              <id>netflix-candidates</id>
              <name>Netflix Candidates</name>
              <url>https://artifactory-oss.prod.netflix.net/artifactory/maven-oss-
          candidates</url>
              <snapshots>
                <enabled>false</enabled>
              </snapshots>
            </repository>
          </repositories>
```

4. Sous le projet Gateway, ajouter la classe **SecurityConfig**

```
package com.example.gatewaye.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.web.reactive.EnableWebFluxSecu
rity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.oauth2.jwt.ReactiveJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusReactiveJwtDecoder;
import org.springframework.security.web.server.SecurityWebFilterChain;

@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

  // Configuration de la chaîne de sécurité pour le Gateway
  @Bean
  public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity
http) {
    return http
        .csrf(ServerHttpSecurity.CsrfSpec::disable) // Désactive CSRF
        .authorizeExchange(exchanges -> exchanges
```

```
                .pathMatchers("/eureka/**").permitAll()  // routes publiques
                .anyExchange().authenticated()        // toutes les autres routes
protégées
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())        // active JWT
            )
            .build();
    }

    // Décodeur JWT basé sur la JWK Set URL de Keycloak
    @Bean
    public ReactiveJwtDecoder jwtDecoder() {
        String jwkSetUri =
"http://localhost:8080/realms/Job_Board_Realm/protocol/openid-
connect/certs";
        return NimbusReactiveJwtDecoder.withJwkSetUri(jwkSetUri).build();
    }
}
```

NB : String jwkSetUrihttp://localhost:8080/realms/Job_Board_Realm/protocol/openid-connect/certs
Job_Board_Realm est le Realm Crée

5. Dans le fichier **application.properties**, ajouter la propriété suivante :

```
spring.application.name=gateway

server.port=8081

spring.cloud.gateway.routes[0].id=candidat-service
spring.cloud.gateway.routes[0].uri=lb://candidat-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/candidats/**

spring.cloud.gateway.discovery.locator.enabled=true


# Keycloak realm config
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8080/realms/Job_Board_Realm
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=http://localhost:8080/realms/Job_Board_Realm/protocol/openid-connect/certs


eureka.client.service-url.defaultZone:http://localhost:8761/eureka/
eureka.client.register-with-eureka=true
spring.main.allow-bean-definition-overriding=true
```

6. Maintenant, nous allons tester l'accès sécurisé vers le MS de G. Candidats.

Exécuter les projets :

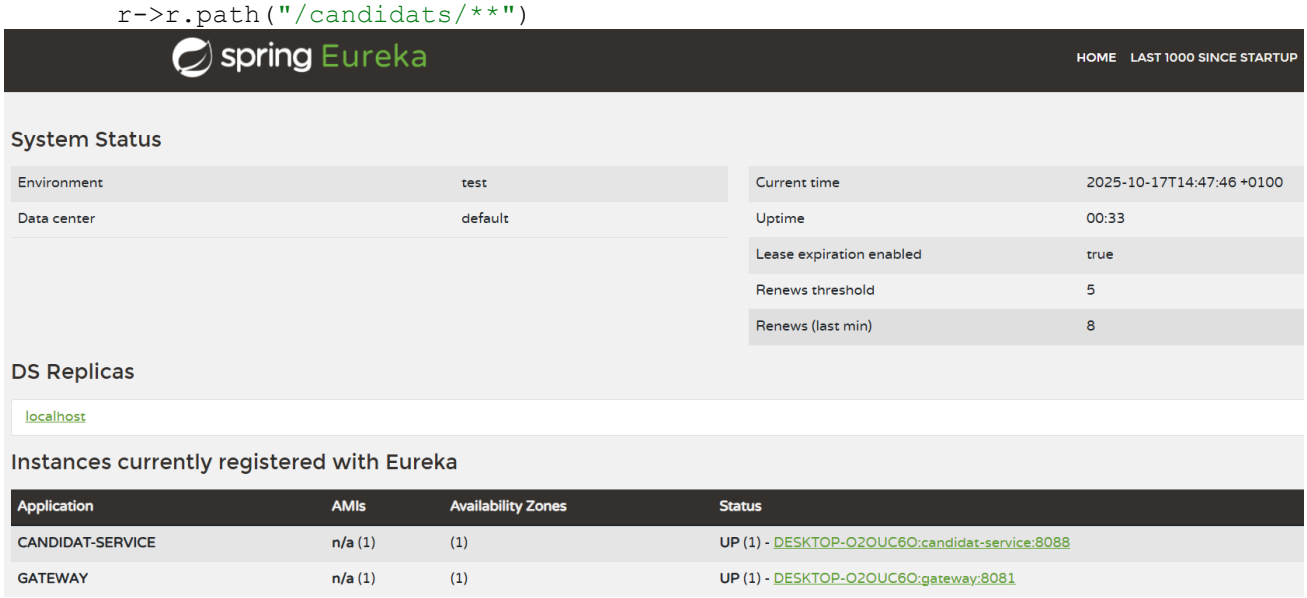
- Gateway
- Candidat
- Eureka

7. Accéder au serveur eureka

NB :

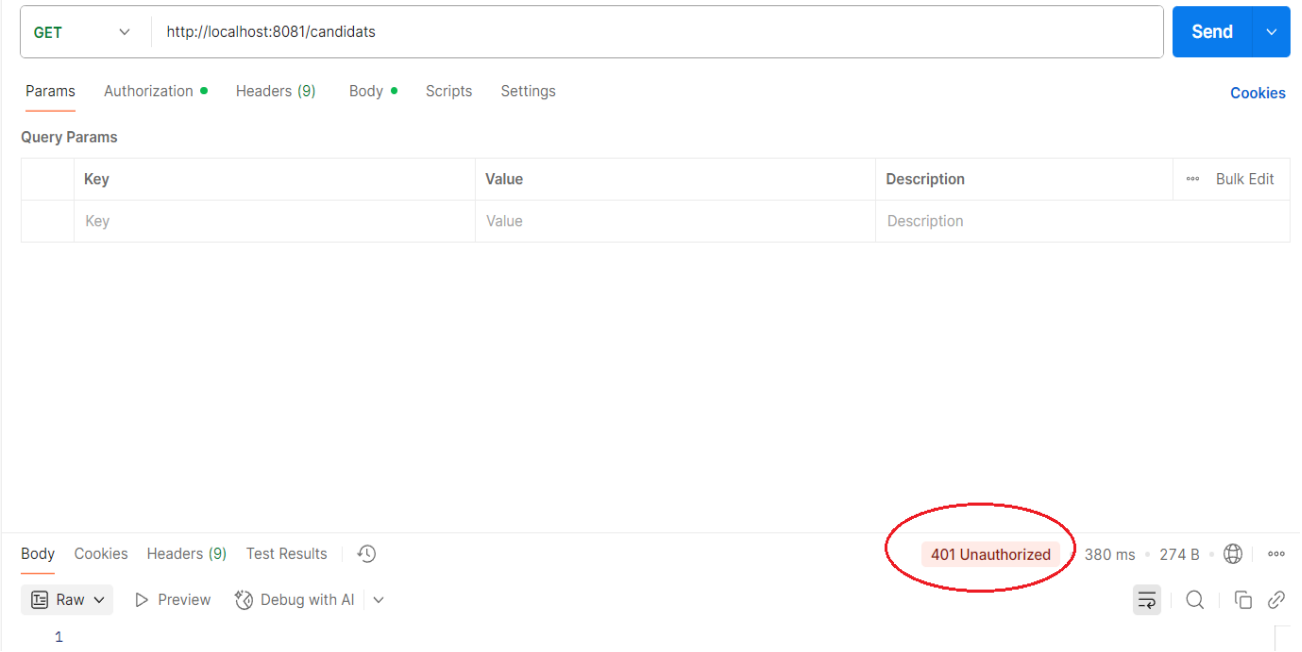
- 8081: est le port de l'API Gateway
- Le path d'accès au MS de Candidat est candidats

```
r->r.path("/candidats/**")
```

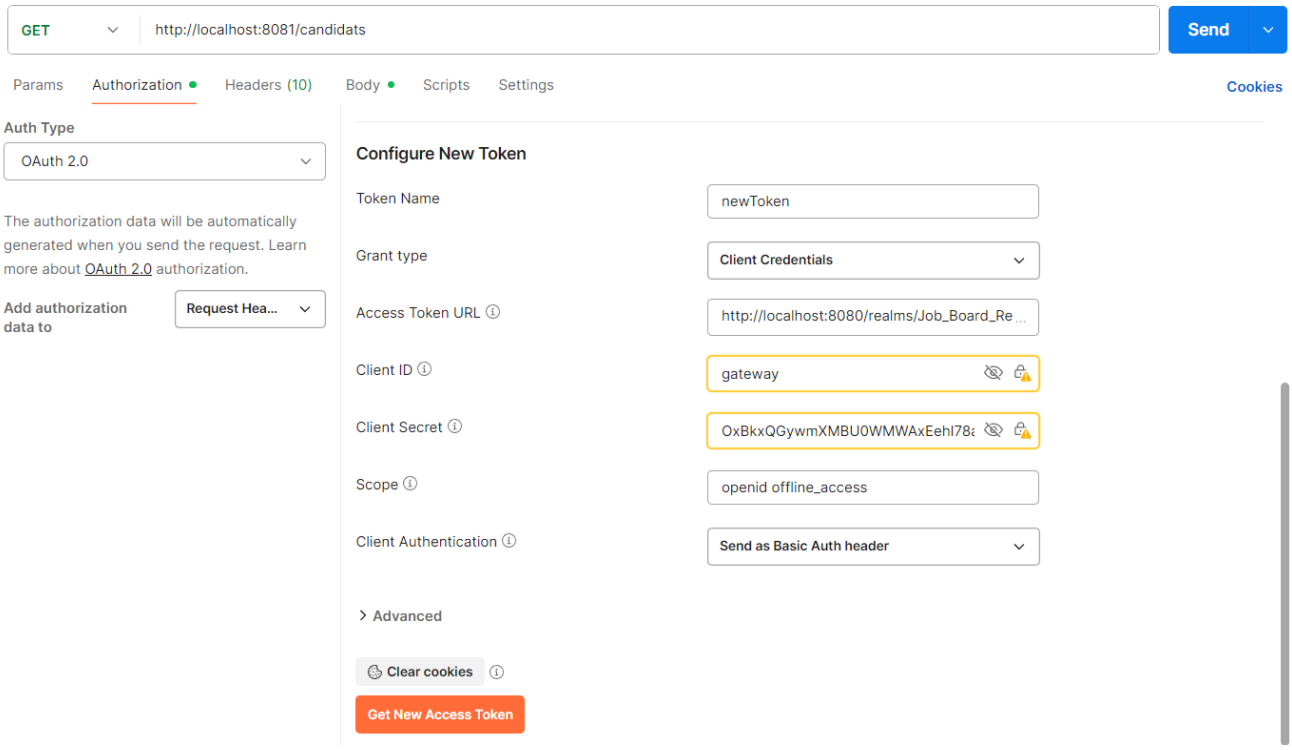


8. Sur le postman, nous allons générer un token à travers le Keycloak
   - Veuillez choisir Authorization-> OAuth 2.0
   - Veuillez configurer un nouveau token

Sans la clé de scurité



Génération de la clé de sécurité



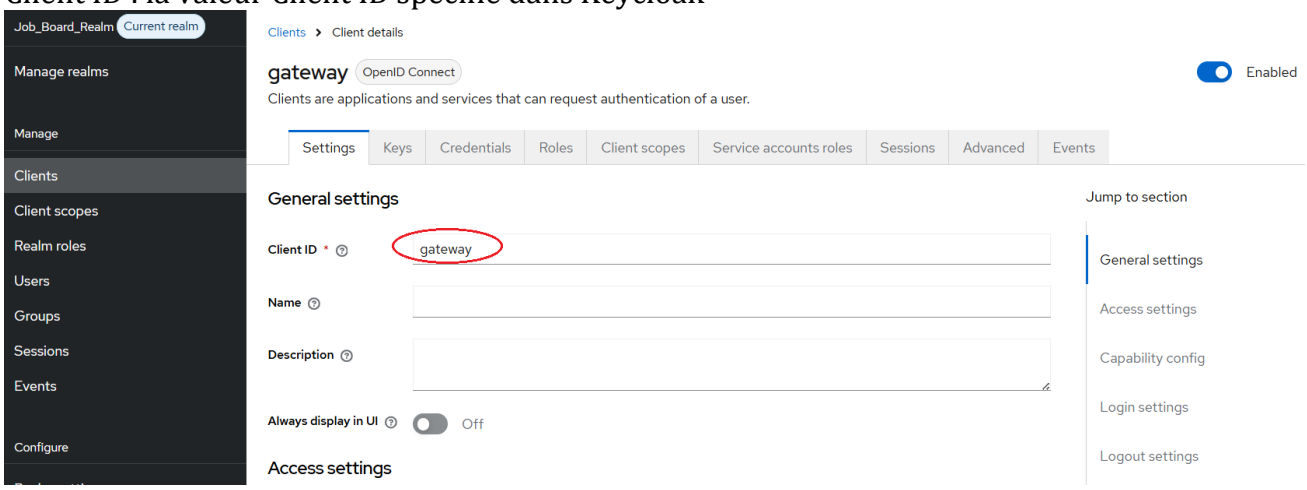
Comme le montre la figure ci-dessus :

- Token Name : veuillez choisir newToken
- Grant type : Client credentials
- Access Token URL : Pour récupérer cette valeur, au niveau du Realm Settings du keycloak, veuillez cliquer sur le lien OpenID EndPoint Configuration. Puis, récupérer la valeur de token-endpoint

---

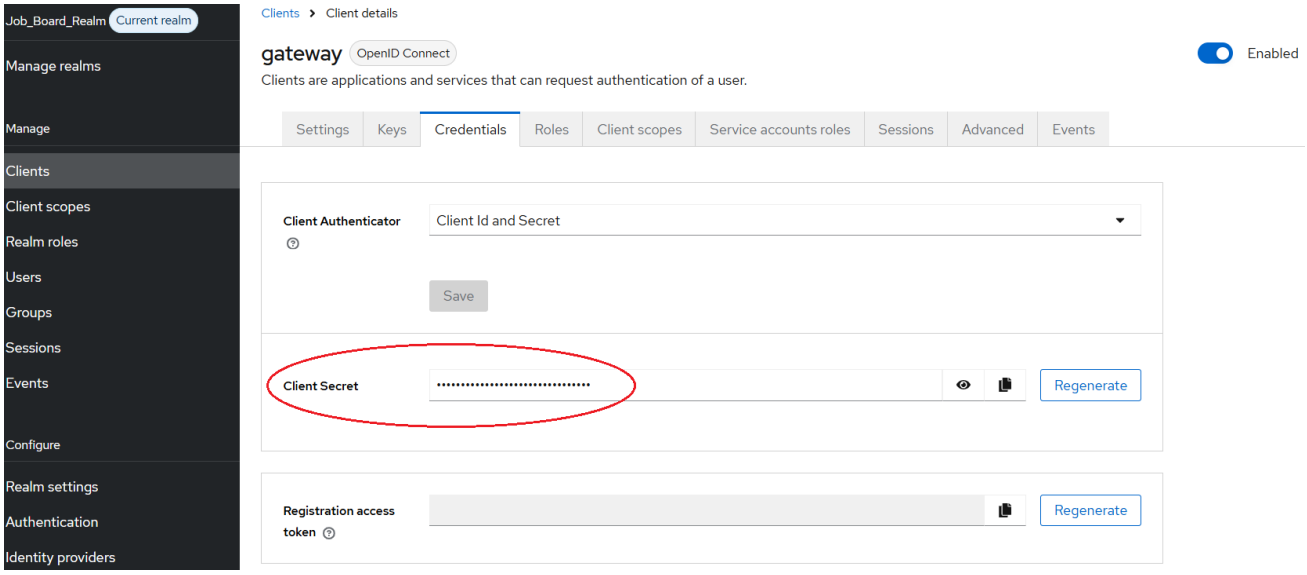
{"issuer":"http://localhost:8080/realms/JobBoardKeycloack","authorization_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-connect/auth","token_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-connect/token","introspection_endpoint":"http://localhost:8080/realms/JobBoardKeycloack/protocol/openid-

- Client ID : la valeur Client ID spécifié dans Keycloak



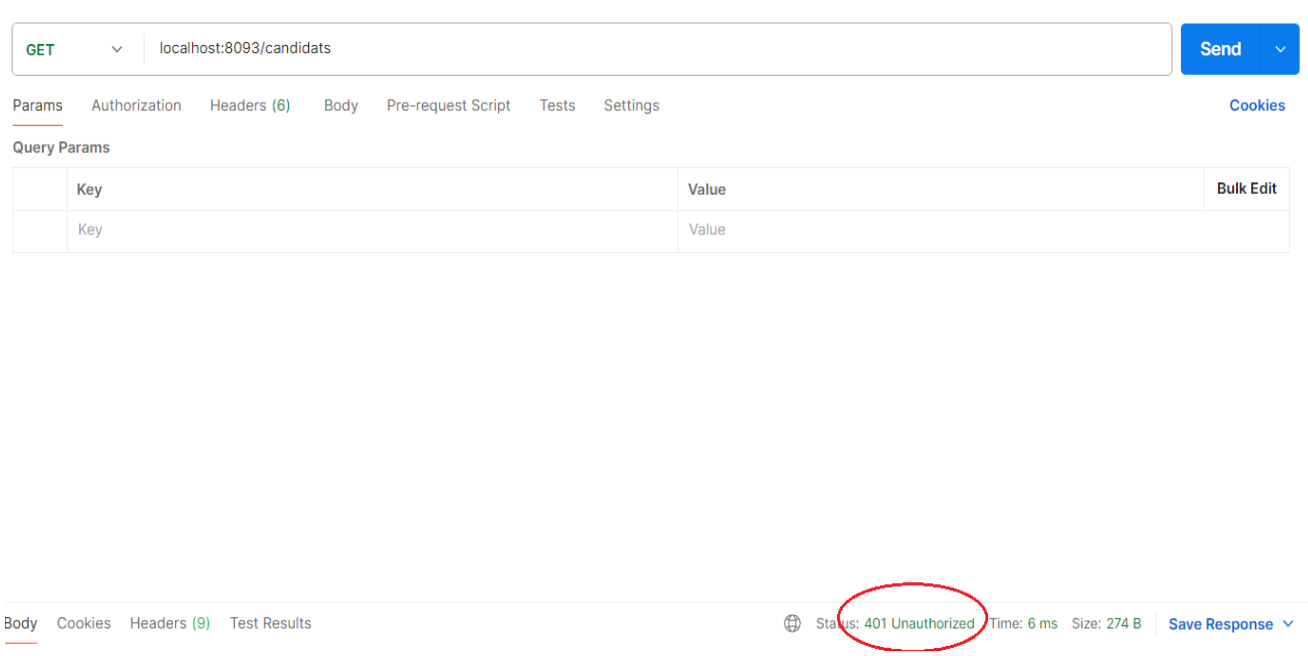
- Client secret :
  Au niveau du Gateway Client, veuillez choisir l'option Credentials. Puis, récupérer la

valeur client secret



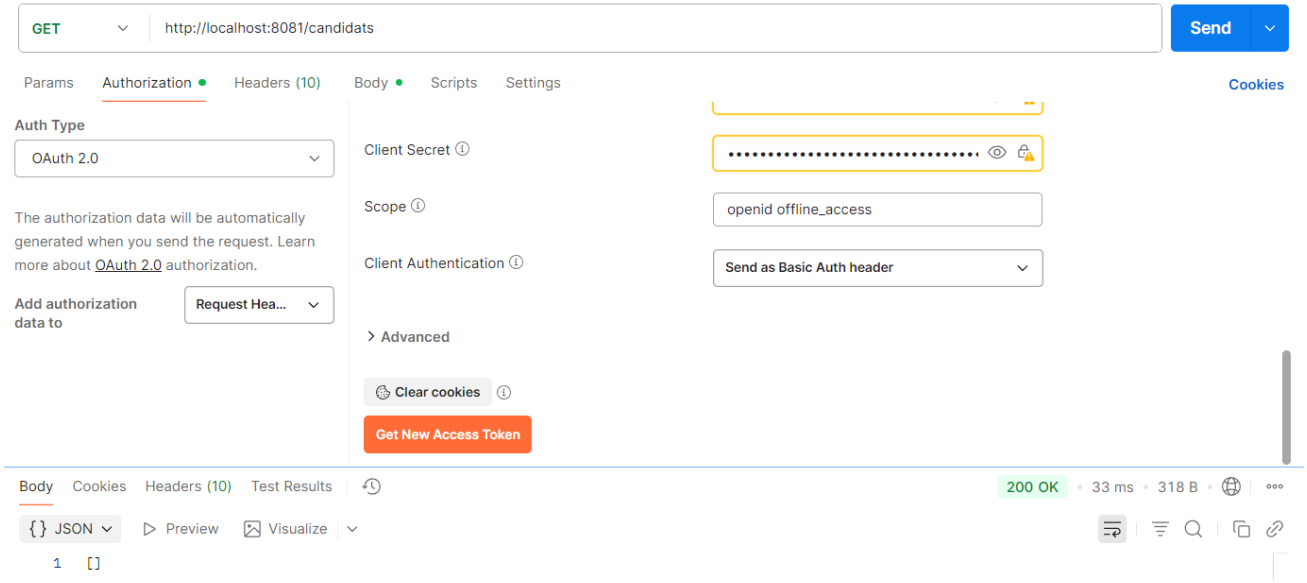
- Scope : openid offline_access
- Client Authentication : Send a Basic Auth header
- Cliquer sur Get New Access Token. Vous pouvez voir la Valeur de token générée.

- Veuillez choisir use token pour utiliser cette valeur comme clé de sécurité.
- Tester l'accès vers le Microservice Candidat comme suit :
  - Sans clé de sécurité



- Avec la clé de sécurité

MANAGE ACCESS TOKENS
All Tokens
Delete
newToken
newToken
newToken
newToken
newToken
Token Details
Use Token
Token Name
newToken
Access Token
eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJ3MlIPTzBITGIIT0k
2OEtiRDlVQmwxN0xOOEJ0cUg5ZmVRMHhtVlpKaXhJln0.eyJleHAiOjE3N
jA3MTI2NDQsImlhdCI6MTc2MDcxMjM0NCwianRpIjoidHJydGNjOjdINjI3
NjA0LTQzODItMmM3Ny05MThlLTgzYWViODE3ZDAxMyIsImlzcyI6Imh0d
HA6Ly9sb2NhbGhvc3Q6ODA4MC9yZWFsbXMvSm9iX0JvYXJkX1JlYWx
tIiwiYXVkIjoiYWNjb3VudCIsInN1YiI6IjlkMzE1OThiLTEyYmUtNDI0NS1iOD
AxLWNlOTgxMzJjYTU2NiIsInR5cCI6IkJlYXJlciIsImF6cCI6ImdhdGV3YXki
LCJhY3IiOiIxIiwiYWxsb3dlZC1vcmlnaW5zIjpbImh0dHA6Ly9sb2NhbGhvc
3Q6ODA4MSJdLCJyZWFsbV9hY2NlcS3MiOnsicm9sZXMiOlsib2ZmbGluZ
V9hY2NlcS3MiLCJ1bWFfYXV0aG9yaXphdGlvbiIsImRlZmF1bHQtcm9sZXM
tam9iX2JvYXJkX3JlYWxtIl19LCJyZXNvdXJjZV9hY2NlcS3MiOnsiZ2F0ZXd
heSI6eyJyb2xlcyI6WyJ1bWFfcHJvdGVjdGlvbiJdfSwiYWNjb3VudCI6eyJy


GET
http://localhost:8081/candidats
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Cookies
Auth Type
OAuth 2.0
The authorization data will be automatically generated when you send the request. Learn more about OAuth 2.0 authorization.
Add authorization data to
Request Hea...
Client Secret
Scope
openid offline_access
Client Authentication
Send as Basic Auth header
Advanced
Clear cookies
Get New Access Token
Body
Cookies
Headers (10)
Test Results
200 OK
33 ms
318 B
JSON
Preview
Visualize
1 []